



Le but du TP est de créer, de représenter et de sortir d'un labyrinthe.

1. MODELISATION D'UN LABYRINTHE

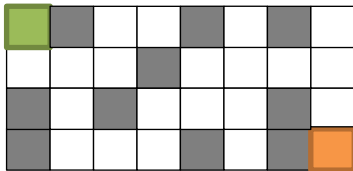
Les labyrinthes peuvent être vus comme des graphes :

Un labyrinthe, composé de hauteur $\times$ largeur cases numérotées par des couples (i, j) , est représenté par un graphe (non orienté) dont les sommets sont étiquetés par des couples (i, j) .

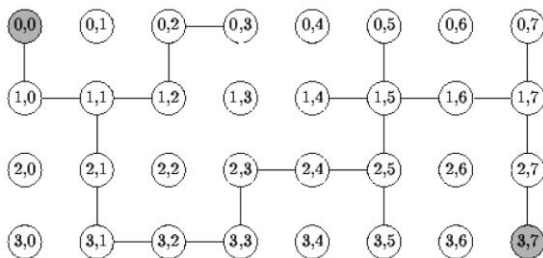
Une case (i_0, j_0) située à la ligne i_0 et à la colonne j_0 communique avec une autre case voisine (i_1, j_1) , si et seulement s'il existe une arête entre les sommets (i_0, j_0) et (i_1, j_1) du graphe.

On distingue deux cases (sommets) particulières (particuliers), ce sont l'entrée (i_e, j_e) et la sortie (i_s, j_s) du labyrinthe.

Soit le labyrinthe **lab** suivant dont l'entrée est $(3, 7)$ et la sortie est $(0, 0)$:



Ce labyrinthe sera alors représenté par le graphe non orienté suivant :



Les parties non accessibles du labyrinthe sont donc représentées par des sommets isolés (qui n'ont pas de voisins).

Pour le reste du TP, on travaillera sur des labyrinthes carrés et on utilisera les classes **Graphe** et **Pile** déjà rencontrées.

1. REPRESENTATION D'UN LABYRINTHE ET D'UNE SOLUTION SOUS TKINTER

Pour tracer les labyrinthes créés, on utilise le module **TKinter**.

Sur l'utilisation de **TKinter**, on se contente ici d'en montrer un exemple qui présente ce qui est strictement nécessaire au TP.

a) Représentation du labyrinthe

Ex 1 : Un Exemple d'utilisation du module se trouve dans le fichier [exemple tkinter.py](#) (disponible sur Moodle).

Faire exécuter cet exemple et repérer les différentes instructions **TKinter** qui entrent en jeu.

1. Dessiner un labyrinthe carré de côté 8 puis créer un graphe (à 64 sommets) qui représente ce labyrinthe. On pourra créer une liste en compréhension pour générer les 64 sommets (de $(0,0)$ à $(7,7)$).
2. Écrire une fonction **represente_laby(canevas,G)** qui prend en paramètre un canvas (canevas) et un graphe (G) et qui affiche le labyrinthe modélisé par ce graphe. Un sommet isolé (pas de voisin) sera représenté par un carré bleu et les sommets non isolés seront représentés par des carrés blancs.
3. Écrire une fonction **represente_entree_sortie(canevas,G,entree,sortie)** qui prend en paramètre un canvas (canevas), un graphe (G), un couple (i_e, j_e) (entrée) et un couple (i_s, j_s) (sortie), qui vérifie que l'entrée et la sortie sont bien des sommets du graphe, puis qui les affiche respectivement en orange et en vert.

b) Représentation d'une solution

4. Écrire une fonction **represente_chemin(canevas,G,entree,sortie)** qui prend en paramètre un canvas et un graphe modélisant un labyrinthe, un couple (i_e, j_e) (entrée) et un couple (i_s, j_s) (sortie), et qui affiche le chemin de l'entrée vers la sortie de ce labyrinthe.
(On pourra par exemple remplir partiellement les sommets visités pour sortir du labyrinthe avec un disque ou un carré de couleur jaune (ou ce que vous voulez qui vous fait plaisir)).
5. Appeler cette fonction lorsque l'on appuie sur la touche "r" (comme "résoudre").

2. UNE PREMIERE AMELIORATION.

Pour faciliter la création du graphe, on propose d'utiliser une matrice pour représenter le labyrinthe puis de "convertir" cette matrice en un graphe.

La matrice sera formée de 0 et de 1 représentant chaque sommet. On notera 1 un sommet accessible (donc ayant des voisins) et 0 un sommet isolé. Ainsi on pourrait associer le premier labyrinthe **lab** à la matrice :

```
[ [1,0,1,1,0,1,0,1] ,
  [1,1,1,0,1,1,1,1] ,
  [0,1,0,1,1,1,0,1] ,
  [0,1,1,1,0,1,0,1] ,
  [0,0,0,0,0,0,0,0] ,
  [0,0,0,0,0,0,0,0] ,
  [0,0,0,0,0,0,0,0] ,
  [0,0,0,0,0,0,0,0] ]
```



Cette matrice n'est donc pas une matrice d'adjacence.

6. Créer une fonction **generation_graphe(M)** qui prend une matrice M en paramètre et qui renvoie le graphe associé à cette matrice (on peut parcourir la matrice en examinant chaque coefficient. Si le coefficient est 0, ce sommet est isolé, et si le coefficient vaut 1 alors il nécessitera la création d'arête(s) avec quelques-uns de ses voisins immédiats).
7. Créer une matrice 8×8 qui représente un labyrinthe (de votre choix mais plutôt complexe) puis générer le graphe associé à cette matrice et en utilisant les fonctions précédentes, afficher et résoudre ce labyrinthe.

3. UNE DEUXIEME AMELIORATION.

Cette deuxième amélioration vise à générer automatiquement et aléatoirement des labyrinthes ayant une solution.

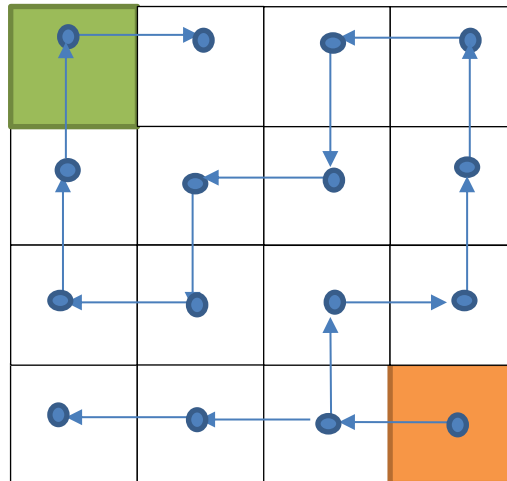
L'idée consiste à :

- Imaginer dans un premier temps, à partir d'une matrice carrée d'ordre n constituée uniquement de 1, un graphe **graphe_sans_les_murs** qui modéliserait un labyrinthe peu labyrinthique puisque sans murs.

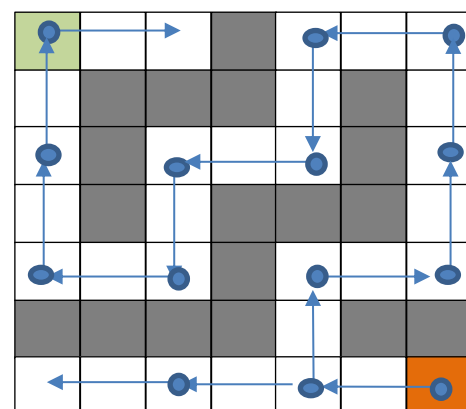
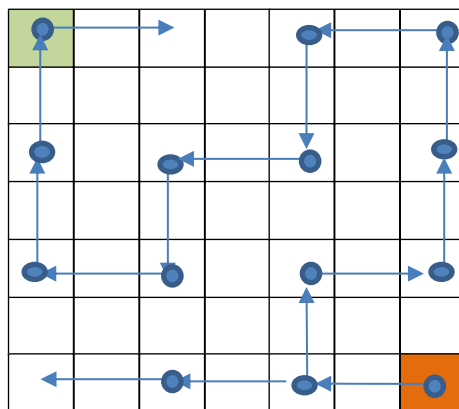
Ce « labyrinthe » serait alors de taille $n \times n$.



- Générer dans ce graphe un parcours aléatoire. Ce graphe étant connexe, il existe un chemin de l'entrée vers la sortie.



- Transplanter ce parcours dans une matrice d'ordre $2n - 1$ afin de pouvoir construire les murs entre deux circulations parallèles du parcours généré ou face à des demi-tours.



On pourra initialiser la matrice en la couvrant de murs, puis en libérant les cases « circulées ».

- Écrire la fonction suivante, dont on donne la *docstring*, et qui constitue une déclinaison de la fonction `parcours_arcs(g,depart,arcs_coches=None)` déjà rencontrée :

```
def parcours_arcs_random(g,depart,arcs_coches=None):
    """renvoie le dictionnaire des arcs parcourus où
    la clé est l'extrémité de l'arc et la valeur est l'origine de l'arc.
    lorsque un choix de chemin est possible, il est réalisé aléatoirement"""
```



On peut utiliser la fonction `shuffle()` du module `random` :
`shuffle(L)` modifie la liste `L` en modifiant uniquement l'ordre de ses éléments.

- Écrire une fonction `dedale(n)` qui prend en paramètre un entier `n` qui représente la taille du labyrinthe si peu labyrinthe avant la création des murs et qui renvoie le graphe à $(2n - 1) \times (2n - 1)$ sommets qui représente le labyrinthe enfin labyrinthe avec ses murs.

10. Les fonctions écrites dans les sections précédentes permettent alors de générer et résoudre des labyrinthes ressemblant à celui-ci (de taille 81) :

